



**Junta de Andalucía**  
Consejería de Educación y Deporte

# UNIDAD DE TRABAJO 9

## ENTRADA Y SALIDA DE INFORMACIÓN

---

### Módulo de Programación



This work is licensed under a [Creative Commons Attribution-NonCommercial-ShareAlike 4.0 International License](https://creativecommons.org/licenses/by-nc-sa/4.0/).

Autor: Fran Gómez

1. Interfaces de entrada y salida
2. Flujo de datos
  - a. Tipos de flujos
  - b. Flujos predeterminados
  - c. Clase Scanner
  - d. Utilización de flujos mediante sus clases
3. Ficheros y directorios
  - a. Tipos
  - b. Creación y eliminación
  - c. Escritura y lectura
  - d. Otras operaciones
4. Serialización
5. XML en Java

# Criterios de Evaluación



| Resultado de Aprendizaje                                                                                                             | % módulo | Criterio de Evaluación                                                                                    | % módulo | %RA | UD                          |
|--------------------------------------------------------------------------------------------------------------------------------------|----------|-----------------------------------------------------------------------------------------------------------|----------|-----|-----------------------------|
| RA5 Realiza operaciones de entrada y salida de información, utilizando procedimientos específicos del lenguaje y librerías de clases | 10       | a) Se ha utilizado la consola para realizar operaciones de entrada y salida de información.               | 1        | 10  | UD10 Ficheros y flujos de   |
|                                                                                                                                      |          | b) Se han aplicado formatos en la visualización de la información.                                        | 1        | 10  | UD10 Ficheros y flujos de   |
|                                                                                                                                      |          | c) Se han reconocido las posibilidades de entrada / salida del lenguaje y las librerías asociadas.        | 1        | 10  | UD10 Ficheros y flujos de   |
|                                                                                                                                      |          | d) Se han utilizado ficheros para almacenar y recuperar información.                                      | 2        | 20  | UD10 Ficheros y flujos de   |
|                                                                                                                                      |          | e) Se han creado programas que utilicen diversos métodos de acceso al contenido de los ficheros.          | 1        | 10  | UD10 Ficheros y flujos de   |
|                                                                                                                                      |          | f) Se han utilizado las herramientas del entorno de desarrollo para crear interfaces gráficas de usuario. | 1        | 10  | UD12 Creación de interfaces |
|                                                                                                                                      |          | g) Se han programado controladores de eventos.                                                            | 1        | 10  | UD12 Creación de interfaces |
|                                                                                                                                      |          | h) Se han escrito programas que utilicen interfaces gráficas para la entrada y salida de información.     | 2        | 20  | UD12 Creación de interfaces |
| RA6 Escribe programas que manipulen información seleccionando y utilizando tipos avanzados de datos                                  | 10       | a) Se han escrito programas que utilicen arrays                                                           | 2        | 20  | UD4 Arrays y cadenas de     |
|                                                                                                                                      |          | b) Se han reconocido las librerías de clases relacionadas con tipos de datos avanzados.                   | 1        | 10  | UD9 Colecciones y TAD       |
|                                                                                                                                      |          | c) Se han utilizado listas para almacenar y procesar información.                                         | 1        | 10  | UD9 Colecciones y TAD       |
|                                                                                                                                      |          | d) Se han utilizado iteradores para recorrer los elementos de las listas.                                 | 1        | 10  | UD9 Colecciones y TAD       |
|                                                                                                                                      |          | e) Se han reconocido las características y ventajas de cada una de la colecciones de datos disponibles.   | 1        | 10  | UD9 Colecciones y TAD       |
|                                                                                                                                      |          | f) Se han creado clases y métodos genéricos.                                                              | 1        | 10  | UD9 Colecciones y TAD       |
|                                                                                                                                      |          | g) Se han utilizado expresiones regulares en la búsqueda de patrones en cadenas de texto.                 | 1        | 10  | UD4 Arrays y cadenas de     |
|                                                                                                                                      |          | h) Se han identificado las clases relacionadas con el tratamiento de documentos XML.                      | 1        | 10  | UD10 Ficheros y flujos de   |
|                                                                                                                                      |          | i) Se han realizado programas que realicen manipulaciones sobre documentos XML.                           | 1        | 10  | UD10 Ficheros y flujos de   |

Los programas deben poder comunicarse con otros programas o bien con el usuario utilizando algún dispositivo



Para realizar esta comunicación se usan interfaces (en un sentido más general):

- **Interfaces de Entrada**
- **Interfaces de Salida**



🔍 Search Google or type a URL



## 1. La Capa Física (Hardware)

Son los dispositivos que puedes tocar.

- **Teclado:** Envía impulsos eléctricos (scancodes).
- **Monitor:** Recibe señales para iluminar píxeles.

## 2. La Capa Lógica (Consola / Terminal)

Es el software del Sistema Operativo que hace de "traductor".

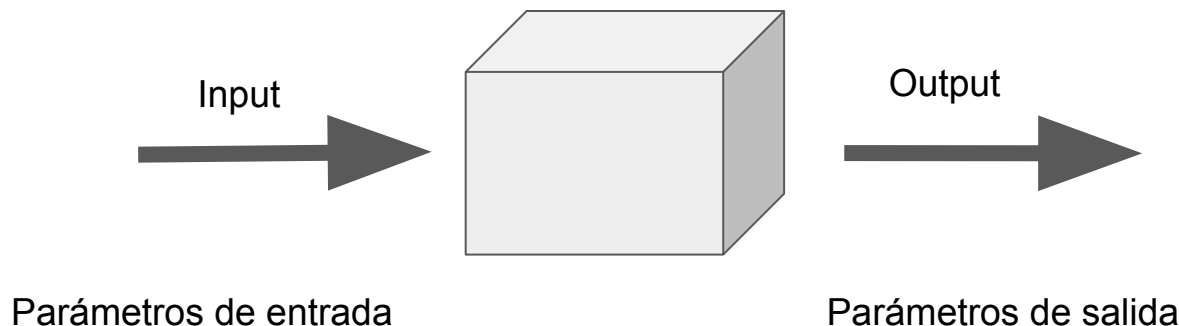
- La **Consola** es una representación virtual. Ella dice: "No me importa qué marca de teclado tengas, yo capturo los caracteres y se los paso al programa que los pida".

## 3. La Capa de Programación (Flujos / Streams)

Aquí es donde vive Java con `System.in` y `System.out`.

- Java no ve teclas ni cables; ve **flujos de datos (Streams)**. Para Java, la consola es solo un "enchufe" de donde vienen o a donde van bytes.

Cualquier **programa** que desee interactuar o trabajar con datos debe tener la capacidad de leer y/o escribir datos.  $\Rightarrow$  **argumentos y parámetros**





# Entrada Y Salida De Información

¿Cómo introducimos datos y cómo los leíamos hasta ahora?

```
class HolaMundo {  
    public static void main (String[] args) {  
        String nombre = "Fran";  
        System.out.println("Hello, World: " + nombre);  
    }  
}
```

Hello, World: Fran

Process finished with exit code 0

```
Scanner sc = new Scanner(System.in);  
try {  
    int dividendo = sc.nextInt();  
    int divisor = sc.nextInt();  
    System.out.println("Resultado: " + (
```

");

Name: HolaMundo

☐ Store as project file

Build and run

[Modify options](#) Alt+M

java 17 SDK of '1daw-prog'

org.ieslosremedios.daw1.prog.ut7.HolaMundo

Fran

Press Alt for field hints

```
class HolaMundo {  
    public static void main (String[] args) {  
        System.out.println("Hello, World: " + args[0]);  
    }  
}
```

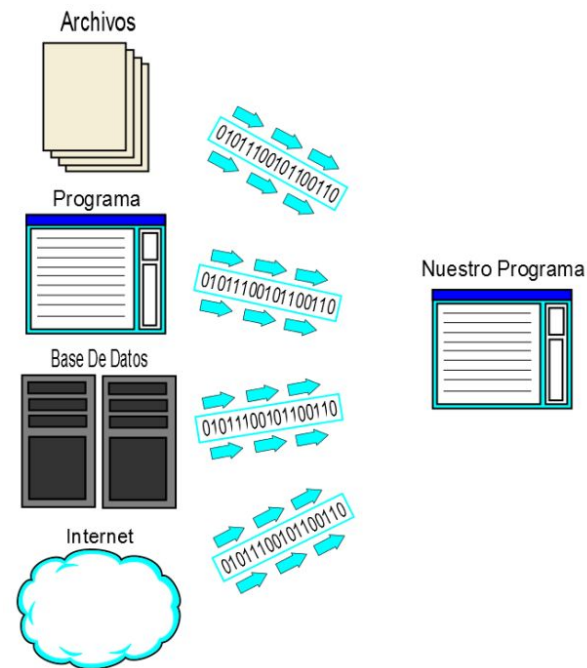
Hello, World: Fran

Process finished with exit code 0

En Java la entrada y salida de datos se lleva a cabo mediante flujos de datos  $\Rightarrow$  **Streams**

**Flujo** de datos  $\Rightarrow$  **Secuencia** ordenada de datos que se transmite en **serie**, es decir, uno detrás de otro, desde un origen hasta un destino

El flujo supone una capa de **abstracción** que nos permite olvidarnos de qué **interfaz** tenemos en el origen o en el destino, en ambos casos usamos el flujo de la misma manera, ya sea escribir en una impresora, leer de un fichero, mostrar algo en la pantalla, etc.





Según el sentido del flujo pueden ser:

- Flujo de **Entrada** ⇒ Se **LEE** información
- Flujo de **Salida** ⇒ Se **ESCRIBE** información

También existen flujos de entrada y salida si se usan en ambos sentidos

Según la forma de acceder al flujo pueden ser:

- Flujo de **acceso directo**  $\Rightarrow$  Se accede directamente al elemento deseado
- Flujo de **acceso secuencial**  $\Rightarrow$  Para acceder al elemento deseado hay que pasar por los anteriores

Según el tipo de datos del flujo pueden ser:

- Flujo de **caracteres** ⇒ los datos se codifican en caracteres. Ej: Unicode o ASCII
- Flujo de **bytes** ⇒ los datos no se codifican. Ejemplo los ficheros binarios.

```
MainEscribirTexto.java fichero_texto.txt ×
1 Esto es la primera línea de texto
2 Esto es otra línea de texto
3 Esto es la última línea de texto
4
```

|      |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |
|------|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|
| 0000 | FF | D8 | FF | E1 | 1D | FE | 45 | 78 | 69 | 66 | 00 | 00 | 49 | 49 | 2A | 00 |
| 0010 | 08 | 00 | 00 | 00 | 09 | 00 | 0F | 01 | 02 | 00 | 06 | 00 | 00 | 00 | 7A | 00 |
| 0020 | 00 | 00 | 10 | 01 | 02 | 00 | 14 | 00 | 00 | 00 | 80 | 00 | 00 | 00 | 12 | 01 |
| 0030 | 03 | 00 | 01 | 00 | 00 | 00 | 01 | 00 | 00 | 00 | 1A | 01 | 05 | 00 | 01 | 00 |
| 0040 | 00 | 00 | A0 | 00 | 00 | 00 | 1B | 01 | 05 | 00 | 01 | 00 | 00 | 00 | A8 | 00 |
| 0050 | 00 | 00 | 28 | 01 | 03 | 00 | 01 | 00 | 00 | 00 | 02 | 00 | 00 | 00 | 32 | 01 |
| 0060 | 02 | 00 | 14 | 00 | 00 | 00 | B0 | 00 | 00 | 00 | 13 | 02 | 03 | 00 | 01 | 00 |
| 0070 | 00 | 00 | 01 | 00 | 00 | 00 | 69 | 87 | 04 | 00 | 01 | 00 | 00 | 00 | C4 | 00 |
| 0080 | 00 | 00 | 3A | 06 | 00 | 00 | 43 | 61 | 6E | 6F | 6E | 00 | 43 | 61 | 6E | 6F |
| 0090 | 6E | 20 | 50 | 6F | 77 | 65 | 72 | 53 | 68 | 6F | 74 | 20 | 41 | 36 | 30 | 00 |
| 00A0 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | B4 | 00 | 00 | 00 |
| 00B0 | 01 | 00 | 00 | 00 | B4 | 00 | 00 | 00 | 01 | 00 | 00 | 00 | 32 | 30 | 30 | 34 |
| 00C0 | 3A | 30 | 36 | 3A | 32 | 35 | 20 | 31 | 32 | 3A | 33 | 30 | 3A | 32 | 35 | 00 |
| 00D0 | 1F | 00 | 9A | 82 | 05 | 00 | 01 | 00 | 00 | 00 | 86 | 03 | 00 | 00 | 9D | 82 |
| 00E0 | 05 | 00 | 01 | 00 | 00 | 00 | 8E | 03 | 00 | 00 | 00 | 90 | 07 | 00 | 04 | 00 |

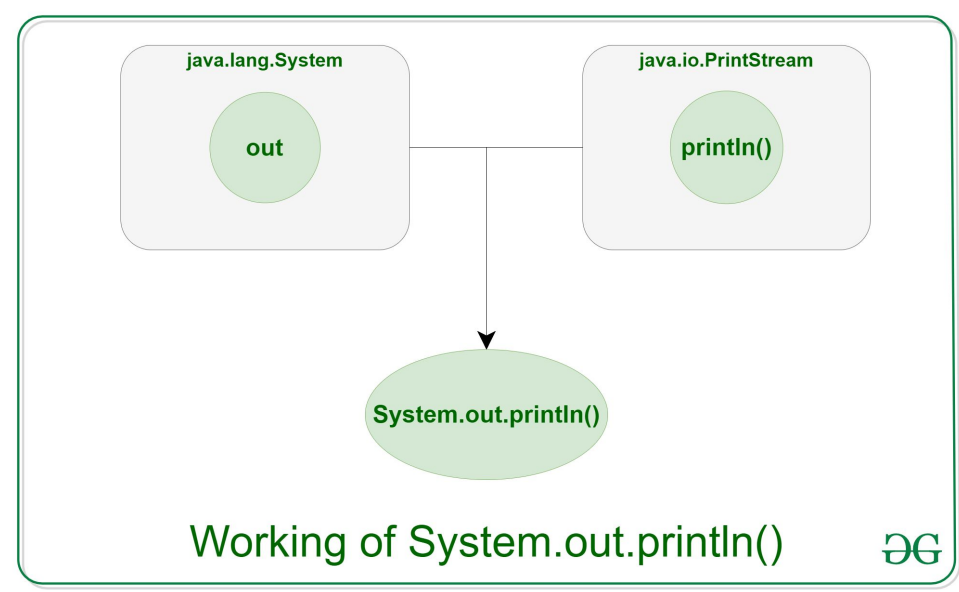
Mientras se ejecuta un programa, se mantienen abiertos algunos flujos de datos para ser usados sin necesidad de crearlos manualmente (como luego veremos).

Estos son:

- `System.out`
- `System.err`
- `System.in`

**PrintStream out**

**InputStream in**



PrintStream (el atributo **out**)

*La salida estándar es la pantalla (consola)*

|                    |                                                                                                                                        |
|--------------------|----------------------------------------------------------------------------------------------------------------------------------------|
| System.out.write   | Escribe los bytes enviados como argumento en el flujo.                                                                                 |
| System.out.flush   | Vacía el flujo por la salida (out). Es necesario hacerlo en el write si no está en automático o el byte no representa una nueva línea. |
| System.out.print   | System.out.write + System.out.flush. Admite diferentes tipos de datos (no bytes como el write).                                        |
| System.out.println | Como el print pero con retorno de carro                                                                                                |
| System.out.printf  | Como el print pero con formato.                                                                                                        |

InputStream (el atributo *in*)

*La entrada estándar es el teclado (consola)*

|                           |                                                                                              |
|---------------------------|----------------------------------------------------------------------------------------------|
| System.in.read ()         | Devuelve el siguiente byte del flujo. Hay que pasarlo al tipo que queramos mediante casting. |
| System.in.read (byte[] b) | Almacena en b los bytes del flujo                                                            |

# Ejercicio 1



*// Pedir al usuario que introduzca 4 caracteres por teclado*

*// 1. Imprimir el primero con el write*

*// 2. Imprimir el segundo con print*

*// 3. Imprimir el tercero con println*

*// 4. Imprimir el cuarto con printf*

No usar Scanner, sino los métodos  
expuestos anteriormente

# Ejercicio 2



Dado el siguiente código:

```
import java.io.FileWriter;

public class ExperimentoFlush {
    public static void main(String[] args) throws Exception {
        // Creamos el escritor hacia un archivo
        FileWriter escritor = new FileWriter("prueba.txt");

        // Escribimos algo
        escritor.write("¿Dónde está mi texto?");

        System.out.println("He escrito en el archivo... ¿o no?");

        // El programa se queda esperando aquí para que no termine
        Thread.sleep(10000); // Espera 10 segundos

        // No hemos puesto ni flush() ni close()
    }
}
```

Escribe y ejecuta el programa. Luego responde:

- ¿Se ha escrito algo en prueba.txt? ¿por qué?
- ¿Cómo lo arreglarías? Hazlo y compruébalo.



# Ejercicio 3



*Permitir al usuario introducir un número indeterminado de caracteres.*

*Primero habrá que imprimir las instrucciones para el usuario: "Introduzca varios caracteres"*

*Luego habrá que añadir qué condición debe introducir el usuario para indicar que ya ha terminado de introducir caracteres, con lo que quedaría algo así: "Introduzca varios caracteres y después pulse intro para finalizar"*

*Lo último que haremos es un hola mundo donde se pida el nombre al usuario, utilizando el código anterior.*

*Ej:    output:"Introduzca su nombre"*

*input: Fran*

*output: Hola Fran!*

# Ejercicio 4



*¿Qué hace el siguiente código? Piénsalo y luego pruébalo para confirmarlo.*

```
public static void main(String args[]) {  
    byte b[] = new byte[5];  
    try {  
        System.in.read(b);  
    } catch (IOException ioe) {  
        System.out.println(ioe);  
    }  
    String s = new String(b);  
    System.out.println(s);  
}
```

Añade una capa más de abstracción para leer.

Más sencillo que usar directamente que los métodos de `InputStream`, que sólo leen bytes sin procesarlos (Unicode). Mientras que `Scanner`:

- Es capaz de usar tipos de datos primitivos (`int`, `String`...)
- Es capaz de leer a nivel de línea (tokens)

Los tokens por defecto serán grupos de caracteres alfanuméricos separados por espacios. Aunque también podrían definirse qué queremos que sea un token, por ejemplo una expresión regular.

| Método                    | Qué lee                                     | Ejemplo de uso                                |
|---------------------------|---------------------------------------------|-----------------------------------------------|
| <code>next()</code>       | Una sola palabra (hasta el primer espacio). | <code>String palabra = sc.next();</code>      |
| <code>nextLine()</code>   | Una frase entera (hasta que pulsan Enter).  | <code>String frase = sc.nextLine();</code>    |
| <code>nextInt()</code>    | Un número entero.                           | <code>int edad = sc.nextInt();</code>         |
| <code>nextDouble()</code> | Un número con decimales.                    | <code>double precio = sc.nextDouble();</code> |

**Ejercicio 5:** Hacer un *hola mundo*, pidiendo al usuario su nombre completo y edad. Controlar que la edad sea un número entero.

## "El Adivino Digital".

### Misión:

1. El programa pregunta: "¿Cuál es tu nombre?". (`nextLine`)
2. El programa pregunta: "¿Cuántos años crees que vivirás?". (`nextInt`)
3. El programa responde: "Hola [nombre], los astros dicen que morirás a los [años + 10] por culpa de un café frío".
4. Ahora pregunta primero los años y luego el nombre ¿qué ha pasado?

`java.io.Console` **extends** `Object` **implements** `Flushable`

- Alternativa a `System.out` que añade algunos métodos interesantes
- No se puede ejecutar desde el IDE sino desde consola.
- Para crearla: *`Console console = System.console();`*
- Métodos:
  - `String readLine()`
  - `char[] readPassword()`
  - `Console printf(String format, Object... args)`

Escribe un programa que pida al usuario su nombre de usuario y contraseña y los compare con el valor de una constante. En caso de que introduzca mal sus credenciales, permitirle 3 intentos.

1. Crear el Stream (En los estándar es automático)
2. Leer-escribir
3. Cerrar el Stream (En los estándar es automático)



Nos permiten **abstraernos** de los detalles del dispositivo donde queramos leer o escribir → Usamos objetos en lugar de acceder directamente a los drivers.

Estos objetos son los flujos → Definidos mediante las clases del paquete:

[java.io](http://java.io)

Según el tipo de información que manejan los flujos, usaremos unas clases u otras:

- Flujos de bytes → InputStream y OutputStream (Clases abstractas)
- Flujos de caracteres → Reader y Writer (Clases abstractas)

- Entrada → [InputStream](#)

|                              |                                                                                                     |
|------------------------------|-----------------------------------------------------------------------------------------------------|
| <code>int read()</code>      | Lee el siguiente byte desde el <code>InputStream</code> . Si retorna -1 no se pueden leer más bytes |
| <code>int available()</code> | Devuelve el número de bytes que se pueden leer del <code>InputStream</code> sin un bloqueo          |
| <code>void close()</code>    | Cierra el <code>InputStream</code>                                                                  |

- Salida → [OutputStream](#)

|                                                                    |                                                                          |
|--------------------------------------------------------------------|--------------------------------------------------------------------------|
| <code>int write(int b)</code><br><code>int write (byte[] a)</code> | Escribe uno o varios bytes en el <code>OutputStream</code>               |
| <code>int flush()</code>                                           | Vacía el flujo de salida actual del <code>OutputStream</code>            |
| <code>void close()</code>                                          | Cierra el <code>OutputStream</code> y escribe lo que haya en ese momento |

- `FileOutputStream`
- `BufferedOutputStream`
- `DataOutputStream`
- `ByteArrayOutputStream`
- `PrintStream`
- `PipedOutputStream`
- `FileInputStream`
- `BufferedInputStream`
- `DataInputStream`
- `ByteArrayInputStream`
- `SequenceInputStream`
- `PipedInputStream`

## Ejercicio 7:

Investiga cada una de estas clases y crea un ejemplo para explicar su funcionamiento al resto de la clase.

Elabora la jerarquía de clases e indica los paquetes a los que pertenecen.

¿Eres capaz de encontrar más flujos a parte de los de la lista?

# Clases para manejar flujos de caracteres



- Entrada → **Reader**

|                           |                                                                                  |
|---------------------------|----------------------------------------------------------------------------------|
| <code>int read()</code>   | Lee el siguiente byte desde el Reader. Si retorna -1 no se pueden leer más bytes |
| <code>long skip(n)</code> | Hace que se omitan los n primeros caracteres                                     |
| <code>void close()</code> | Cierra el InputStream                                                            |

- Salida → **Writer**

|                                                                                                         |                                                        |
|---------------------------------------------------------------------------------------------------------|--------------------------------------------------------|
| <code>int write(int c)</code><br><code>int write (String s)</code><br><code>int write (char[] c)</code> | Escribe uno o varios bytes en el OutputStream          |
| <code>int flush()</code>                                                                                | Vacía el flujo de salida actual del Writer             |
| <code>void close()</code>                                                                               | Vacía el flujo de salida actual del Writer y lo cierra |

- `FileWriter`
- `BufferedWriter`
- `CharArrayWriter`
- `PrintWriter`
- `StringWriter`
- `PipedWriter`
- `FileReader`
- `BufferedReader`
- `LineNumberReader`
- `CharArrayReader`
- `StringReader`
- `PipedReader`

## Ejercicio 8:

Investiga cada una de estas clases y crea un ejemplo para explicar su funcionamiento al resto de la clase.

Elabora la jerarquía de clases e indica los paquetes a los que pertenecen.

Añade al ejemplo la utilización de los métodos `mark` y `reset`, explicando para qué los usas.

- Permiten persistencia de datos →

Almacenar los datos para recuperarlos en sucesivas ejecuciones del programa.

|                                                                                                       |                  |                        |      |
|-------------------------------------------------------------------------------------------------------|------------------|------------------------|------|
|  .git                | 11/04/2023 9:57  | File folder            |      |
|  .idea               | 27/03/2023 19:18 | File folder            |      |
|  org                 | 27/03/2023 17:24 | File folder            |      |
|  out                 | 27/03/2023 17:30 | File folder            |      |
|  .gitattributes      | 27/03/2023 17:24 | Git Attributes Sour... | 1 KB |
|  .gitignore          | 27/03/2023 17:30 | Git Ignore Source ...  | 1 KB |
|  1daw-prog-22_23.iml | 27/03/2023 17:26 | IML File               | 1 KB |
|  README.md           | 27/03/2023 17:24 | Markdown Source...     | 1 KB |

# Ficheros y Directorios: Tipos de ficheros



| Texto                   | Binario                     |
|-------------------------|-----------------------------|
| Legibles para un humano | Legibles para los programas |
| Acceso secuencial       | Acceso directo y secuencial |

Conjunto de datos que deben ser accedidos juntos.

Ej: un objeto.

Instancia de la clase Persona



A large light-gray circle represents an instance of the 'Persona' class. Inside the circle, four attributes are listed, each followed by a text input field containing its value:

|                  |           |
|------------------|-----------|
| <b>nombre</b>    | Juan      |
| <b>apellido1</b> | Gil       |
| <b>apellido2</b> | Torres    |
| <b>dni</b>       | 12345678L |

Se guarda en un  
archivo de tipo  
binario



## Clase java.io.**File**

- Constructores
  - *File (String pathname)*
- Métodos
  - *boolean createNewFile()*
  - *boolean mkdir()*
  - *boolean delete()*

La clase File representa un fichero en Java. No lo abre. No lo crea. Sólo lo representa. Sí podemos obtener información sobre él.

# Ficheros y Directorios: Ejemplo



```
File archivo = new File("notas.txt");  
if (archivo.exists()) {  
    System.out.println("Archivo encontrado: " + archivo.getName());  
    System.out.println("Ruta absoluta: " + archivo.getAbsolutePath());  
} else {  
    System.out.println("El archivo no existe.");  
}
```

- Acceso secuencial
  - FileReader, FileInputStream
    - (String **path**)
    - (File **file**)
  - FileWriter, FileOutputStream
    - (String **path** [, boolean **append**])
    - (File **file** [, boolean **append**])
- Acceso directo
  - RandomAccessFile
    - (String **path** ,String **modo**)
    - (File **file** ,String **modo**)

En el constructor se puede indicar tanto la ruta como el objeto File.

En el caso de escritura, también si añadimos o sobrescribimos.

El modo de acceso:

- r → lectura
- rw → lectura y escritura

# Ejemplo FileWriter y FileReader



```
String nombreArchivo = "ejemplo.txt";

// 1. ESCRIBIR en el archivo
try (FileWriter escritor = new FileWriter(nombreArchivo)) {
    escritor.write("Hola desde Java");
    System.out.println("Archivo escrito.");
} catch (IOException e) {
    e.printStackTrace();
}

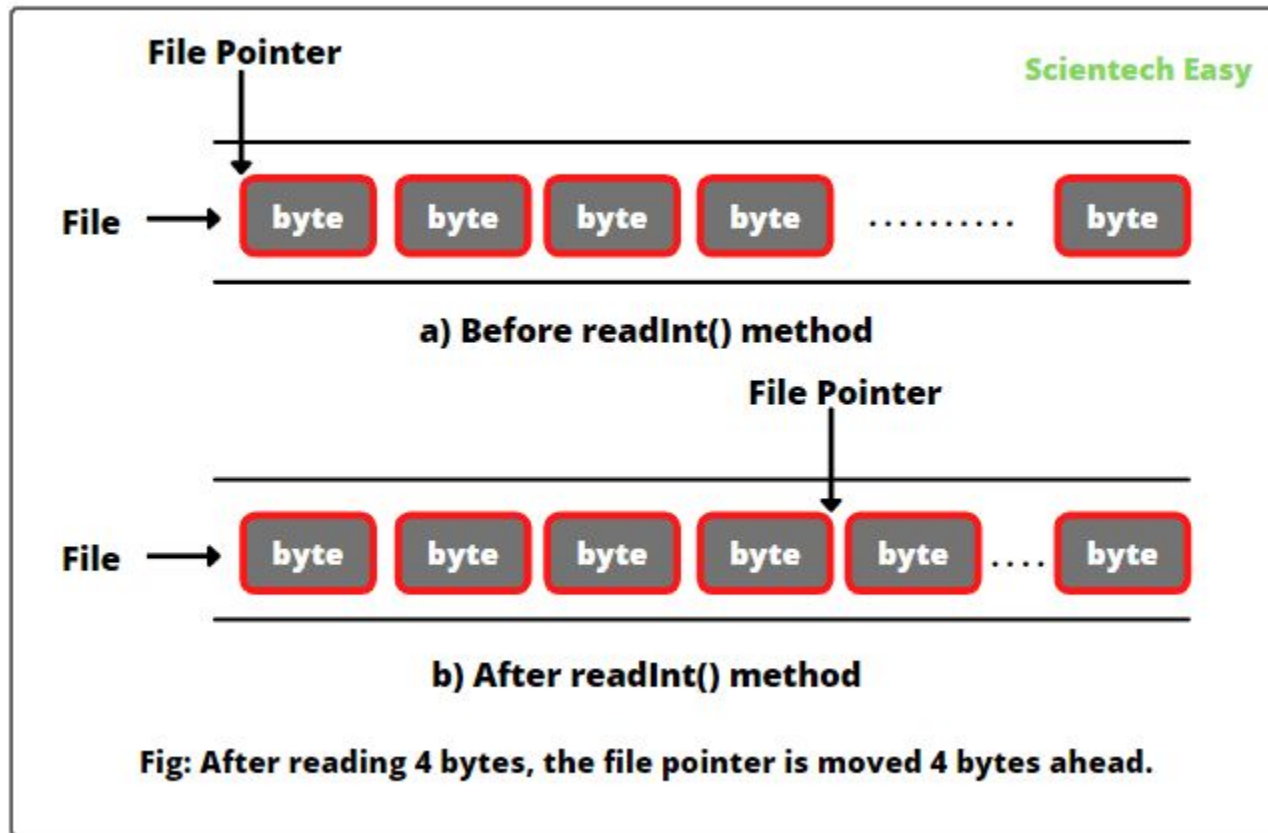
// 2. LEER del archivo
try (FileReader lector = new FileReader(nombreArchivo)) {
    int i;
    System.out.print("Contenido leído: ");
    // Lee carácter por carácter hasta el final (-1)
    while ((i = lector.read()) != -1) {
        System.out.print((char) i);
    }
} catch (IOException e) {
    e.printStackTrace();
}
```

# Ejemplo FileOutputStream y FileInputStream



```
// 1. ESCRIBIR bytes (crear un archivo con datos)
try (FileOutputStream salida = new FileOutputStream("datos.bin")) {
    byte[] datos = {72, 111, 108, 97}; // "Hola" en código ASCII
    salida.write(datos);
    System.out.println("Archivo binario creado.");
} catch (IOException e) {
    e.printStackTrace();
}

// 2. LEER bytes
try (FileInputStream entrada = new FileInputStream("datos.bin")) {
    int byteLeido;
    System.out.print("Bytes leídos: ");
    while ((byteLeido = entrada.read()) != -1) {
        System.out.print(byteLeido + " "); // Muestra los números
    }
} catch (IOException e) {
    e.printStackTrace();
}
```



Se utiliza la clase [java.io](https://docs.oracle.com/javase/7/docs/api/java/io/RandomAccessFile.html).

**RandomAccessFile**

Trata el archivo como un **array de bytes** gigante. Tiene un **puntero** para desplazarse.

## Métodos:

- **seek** → Desplaza puntero del archivo a la posición indicada (empezando en 0 como los arrays)
- **read** → Lee un byte
- **write** → Escribe un byte
- **readTipo/writeTipo** → Lee/Escribe un dato del tipo primitivo indicado.
- **length** → Longitud del fichero

Nota: cada tipo de dato ocupa un número de bytes distinto en Java. Ej: char ocupa 2 bytes en Unicode.

# RandomAccessFile Ejemplo



```
// "rw" significa modo Read (Lectura) y Write (Escritura)
RandomAccessFile archivo = new RandomAccessFile("datos.txt", "rw");

// 1. Escribimos algo inicial
archivo.writeBytes("Hola Mundo");

// 2. Queremos cambiar la 'M' por una 'P'
// 'H' es pos 0, 'o' es 1, 'l' es 2, 'a' es 3, ' ' es 4, 'M' es 5.
archivo.seek(5);
archivo.writeBytes("P");

// 3. Volvemos al principio para leer el resultado
archivo.seek(0);
System.out.println("Resultado: " + archivo.readLine()); // Imprimirá "Hola Pundo"

archivo.close();
```



## Punto de Partida: ¿Qué sabemos ya?



### Streams de E/S

Recordamos que los **Streams** son flujos de datos (bytes o caracteres) que conectan nuestro programa con el exterior.



### Objetos en RAM

Nuestras instancias viven en la memoria volátil. Si el programa se cierra, el objeto desaparece para siempre.



### Archivos

Sabemos escribir texto plano, pero... ¿cómo guardamos una estructura compleja con atributos y tipos?

## El Problema del "Save Game"

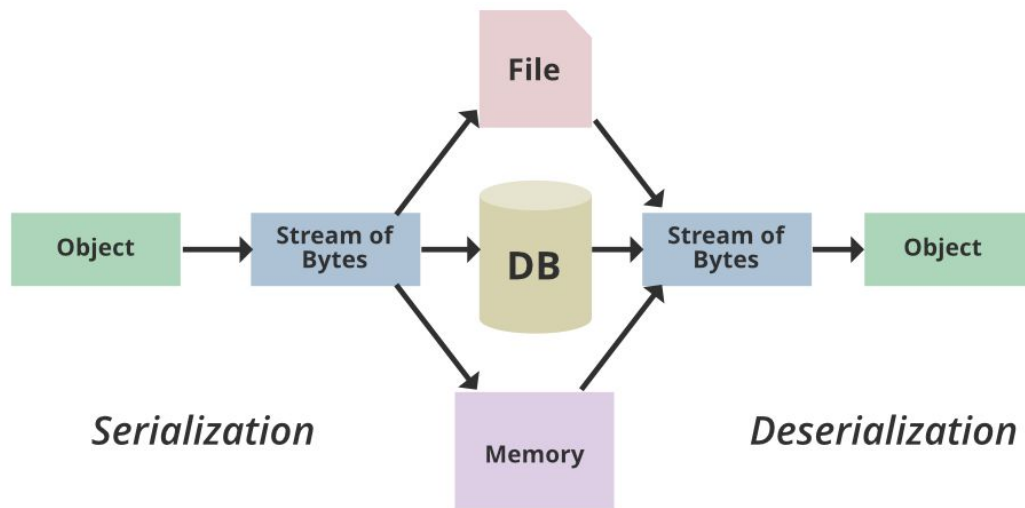
### ¿Cómo se guarda tu progreso?

Imagina un juego de rol: tienes vida, inventario, posición y nivel. No puedes guardar esto como un simple "texto".

Necesitas "congelar" el objeto entero y guardarlo en el disco para que, al volver, todo esté exactamente igual.

**¡Eso es Serializar!**





Fuente: GeeksforGeeks

Proceso mediante el que convertimos objetos a secuencias de bytes para así poder transmitirlos (por ejemplos a un fichero binario) y luego poder volver a transformarlo en un objeto mediante un proceso de deserialización.



**Proceso:** Convertir el estado de un objeto en una secuencia de bytes.



**Uso:** Almacenar en disco, enviar por red o guardar en base de datos.



**Importancia:** Mantiene la integridad de los datos y las relaciones entre objetos.

Para que un objeto se serializable, su clase tiene que cumplir:

- Implementar la interfaz **Serializable**: Es una interfaz marcadora (no tiene métodos)
- **Atributos**: Todos los atributos deben ser serializables también (**primitivos** o clases que implementen **Serializable**).
- Declarar el atributo estático privado **serialVersionUID**: Un ID único para asegurar que la clase que guarda es la misma que la que lee.

- Los atributos que no queramos serializar deben marcarse con el modificador “transient”
  - Ejemplo: A veces hay datos sensibles (passwords) o temporales (un timer) que no deben persistir por eficiencia.
    - `private transient String password;`
- Los atributos estáticos tampoco se serializan, su valor es el mismo para todos los objetos de la clase.

# Ejemplo clase Serializable



```
public class Persona implements Serializable {  
  
    // Recomendado para control de versiones de la clase  
    private static final long serialVersionUID = 1L;  
  
    // Atributos serializables  
    private String nombre;  
    private int edad;  
    private LocalDate fechaNacimiento;  
  
    // No se serializa (aunque sea serializable)  
    private transient String password;  
  
    // No serializable (Object no implementa Serializable)  
    // Debe ser transient para evitar error  
    private transient Object conexion;  
}
```



# Serialización: métodos

- Serializar → **ObjectOutputStream**
  - writeObject → Escribe un objeto en el flujo
- Deserializar → **ObjectInputStream**
  - readObject → Lee un objeto (como tipo Object) del flujo

# Serializar: ObjectOutputStream



```
// 1. Crear objeto
Persona miPersona = new Persona(
    "Fran",
    30,
    LocalDate.of(1995, 5, 10),
    "1234"
);

// 2. Guardar en fichero
try {
    FileOutputStream fichero = new FileOutputStream("persona.dat");
    ObjectOutputStream out = new ObjectOutputStream(fichero);

    out.writeObject(miPersona);

    out.close();

    System.out.println("Persona guardada correctamente");
} catch (IOException e) {
    System.out.println("Error al guardar");
}
```

**Serializable** → “permite guardar el objeto”

**transient** → “este atributo NO se guarda”

Si un atributo **no es serializable** → debe ser **transient**

El fichero **.dat** contiene datos **binarios**, no texto





# Deserializar: ObjectInputStream

```
try {
    // 1. Abrir el fichero
    FileInputStream fichero = new FileInputStream("persona.dat");

    // 2. Crear el flujo de lectura de objetos
    ObjectInputStream in = new ObjectInputStream(fichero);

    // 3. Leer el objeto (hay que hacer casting)
    Persona p = (Persona) in.readObject();

    // 4. Cerrar flujo
    in.close();

    // 5. Mostrar resultado
    System.out.println("Persona leída:");
    System.out.println(p);

} catch (IOException e) {
    System.out.println("Error de entrada/salida");
} catch (ClassNotFoundException e) {
    System.out.println("Clase no encontrada");
}
```

**Los atributos normales** → se recuperan correctamente

**Los *transient*** → *no se guardan*, por eso vuelven como valores por defecto

**Los no serializables** → si no son *transient*, el programa falla al guardar

- Lenguaje de marcado usado para estructurar datos.
  - Datos + Estructura
- Independiente de plataforma y lenguaje.
  - Es un estándar
- Muy usado en configuración, intercambio de datos, etc.
  - Human readable

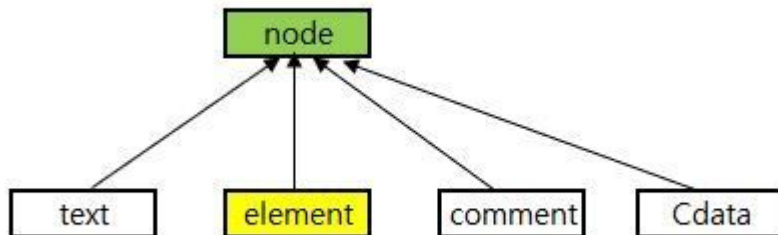
¿Por qué creéis que XML se sigue usando en muchas aplicaciones a pesar de existir otros formatos como JSON?

```
<?xml version="1.0"?>
<quiz>
  <qanda seq="1">
    <question>
      Who was the forty-second
      president of the U.S.A.?
    </question>
    <answer>
      William Jefferson Clinton
    </answer>
  </qanda>
  <!-- Note: We need to add
  more questions later.-->
</quiz>
```

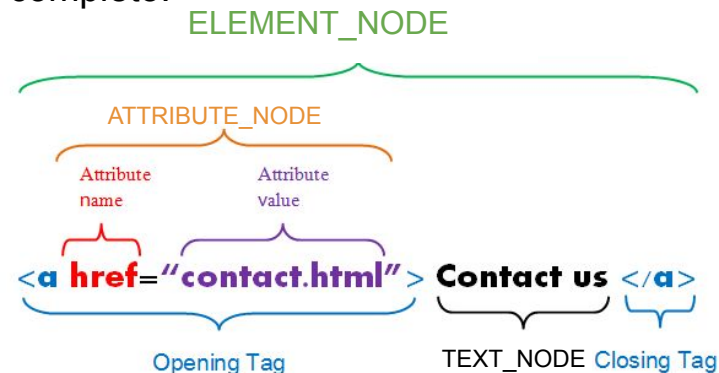
**XML**

- Modelo utilizado para representar ficheros tipo XML o derivados como HTML.
- No depende del lenguaje, se utiliza en cualquier lenguaje
- Existen diferentes librerías o API's para manejarlo

Node → Cualquier objeto en el documento



Element → Un tipo de node que representa a un tag completo.





# DOM: Document Object Model

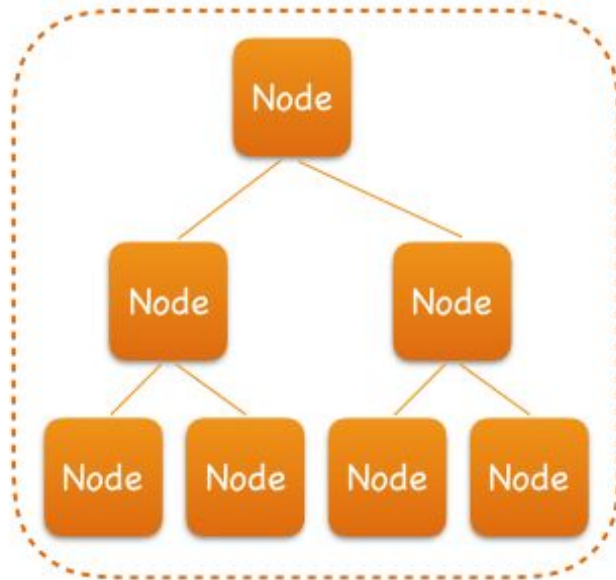
- Los nodos guardan una estructura jerarquizada, en forma de árbol.
- Por tanto tenemos: nodo raíz, nodo padre, nodo hijo, nodo hoja.

## XML document

```
<?xml version="1.0" encoding="UTF-8"?>
<students>
  <student id="001">
    <name>Tom</name>
    <gender>male</gender>
  </student>
  <student id="002">
    <name>Jerry</name>
    <gender>male</gender>
  </student>
</students>
```

*Document Object Model (DOM)*

## Document object



# XML: Transmitir información

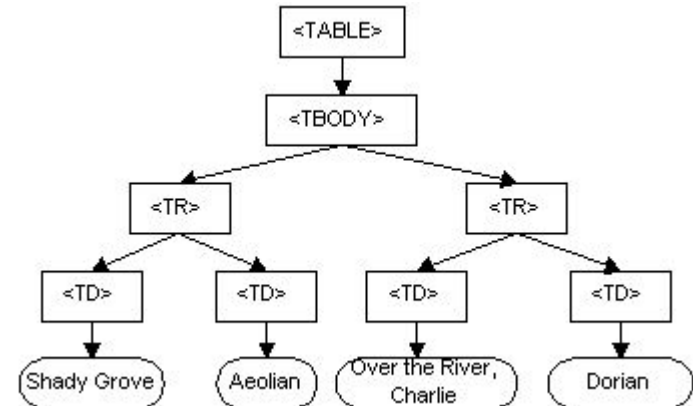


XML

Deserializar o Parsear

```
<TABLE>
  <TBODY>
    <TR>
      <TD>Shady Grove</TD>
      <TD>Aeolian</TD>
    </TR>
    <TR>
      <TD>Over the River, Charlie</TD>
      <TD>Dorian</TD>
    </TR>
  </TBODY>
</TABLE>
```

DOM



Serializar

## DOM (Document Object Model)

- **Concepto:** Carga todo el XML en memoria y crea un árbol de nodos.
- **Cuándo usarlo:** XMLs pequeños/medianos donde necesitas modificar el contenido y guardarlo de vuelta.

## SAX (Simple API for XML)

- **Concepto:** Parseo basado en eventos. Lee el archivo secuencialmente de arriba a abajo sin cargarlo entero en memoria.
- **Cuándo usarlo:** Archivos XML gigantescos (de gigabytes) donde DOM provocaría un error de `OutOfMemoryError`.

Veremos la API DOM → Se trata de una API, es decir, una colección de interfaces que aporta una especificación (el qué), y que son implementadas por una librería de clases (el cómo), normalmente Xerces.

- Librería incluida en JDK
- Paquete [org.w3c.dom](http://org.w3c.dom)

## Principales interfaces

| Tipos Node | Qué representa                                                         | Posibles nodos hijo    |
|------------|------------------------------------------------------------------------|------------------------|
| Document   | Representa todo el documento XML. Conceptualmente es la raíz del árbol | Element, Comment       |
| Node       | Cualquier objeto en el XML (un texto, un tag, un atributo...)          |                        |
| Element    | Representa un elemento (delimitado por etiquetas de principio y fin)   | Element, Text, Comment |
| Attr       | Atributo de un objeto Element                                          | Text                   |
| Comment    | Comentarios                                                            | --                     |
| Text       | Texto                                                                  | --                     |
| NodeList   | TAD que representa una lista de nodos                                  |                        |



## Principales métodos

| Interfaz | Método                               | Descripción                                                                                                     |
|----------|--------------------------------------|-----------------------------------------------------------------------------------------------------------------|
| Element  | getDocumentElement()                 | Accede al nodo raíz del documento. Devuelve un Element.                                                         |
| Element  | getElementsByTagName(String tagname) | Accede a todos los elementos de la etiqueta indicada. Devuelve un NodeList.                                     |
| Node     | getNodeType()                        | Devuelve un número entero indicando el tipo de nodo:<br>1 → ELEMENT_NODE<br>2 → ATTRIBUTE_NODE<br>3 → TEXT_NODE |
| NodeList | getLength()                          | Número de nodos                                                                                                 |
| NodeList | item(int index)                      | Accede a un node por su índice                                                                                  |

## Principales métodos

| Interfaz | Método                      | Descripción                                              |
|----------|-----------------------------|----------------------------------------------------------|
| Element  | getAttribute(String name)   | Devuelve String con el valor del atributo                |
| Node     | getTextContent()            | Devuelve String con el texto del nodo tipo Element       |
| Document | createElement(String name)  | Crea un nodo de tipo elemento                            |
| Text     | createTextNode(String name) | Crea un nodo de tipo texto                               |
| Node     | appendChild(Node node)      | Añade un nodo como hijo del nodo del parámetro implícito |

```
// Cargamos fichero que vamos a leer
File file = new File( pathname: "org/ieslosremedios/daw1/prog/ut7/xml/ejemplo.xml");
```

```
// Parseamos el fichero al Document
DocumentBuilderFactory factory = DocumentBuilderFactory.newInstance();
DocumentBuilder builder = factory.newDocumentBuilder();
Document document = builder.parse(file);
```

```
// Accedemos a todos los nodos con el tag "estudiante"
NodeList estudiantes = document.getElementsByTagName("estudiante");
// Recorremos todos esos nodos
for (int i = 0; i < estudiantes.getLength(); i++) {
    Node nodeEstudiante = estudiantes.item(i);
    // Filtramos todos los que son nodos de tipo elemento
    if (nodeEstudiante.getNodeType() == Node.ELEMENT_NODE) {
        Element elementEstudiante = (Element) nodeEstudiante;
        System.out.println("Nombre del estudiante: " + elementEstudiante.getTextContent());
    }
}
```



# Escritura de XML

```
// Creamos el documento vacío para añadirle a continuación los nodos
// En este caso lo hago todo en una sola línea
Document document = DocumentBuilderFactory.newInstance().newDocumentBuilder().newDocument();
```

```
// Creamos el nodo raíz
Element estudiantes = document.createElement( tagName: "estudiantes");
// Hacemos que cuelgue del documento (estructura de árbol)
document.appendChild(estudiantes); // Creamos el primer nodo y lo colgamos de su padre, el nodo raíz. -->
Element estudianteFran = document.createElement( tagName: "estudiante");
estudiantes.appendChild(estudianteFran);
```

```
// Creamos un nodo de texto que será el valor del elemento anterior
Text fran = document.createTextNode( data: "Fran");
// y lo colgamos del nodo anterior --> <estudiante>Fran</estudiante>
estudianteFran.appendChild(fran);
```

```
// Clases necesarias para finalizar la creación del archivo XML
TransformerFactory transformerFactory = TransformerFactory.newInstance();
Transformer transformer = transformerFactory.newTransformer();
DOMSource source = new DOMSource(document);
StreamResult result = new StreamResult(new File( pathname: "org/ieslosremedios/daw1/prog/ut7/xml/otro.xml"));

// Se realiza la transformación, de Document a Fichero.
transformer.transform(source, result);
```

1. Portfolio: revisar que estén los ejercicios de clase y vuestros apuntes.
2. Proyecto: incorporar elementos de la unidad en el proyecto.
3. Ejercicios de refuerzo y ampliación: para quien quiera hacer más.
4. Examen
5. Feedback